data-science.online

STREAMING SUBSCRIPTION CHURN RISK ANALYSIS: A DECISION TREE APPROACH

Prepared by:
Floriand BESHKU

Connecticut
USA

2026

PAGE INDEX

Project Summary 3

Business Context & Problem Definition.....4

Data Description & Preparation.....5

Methodology (Decision Tree Approach).....6

Analysis & Model Results.....9

Business Implications & Recommendation.....14

Appendix (Python Codes) 15

PROJECT SUMMARY

This project demonstrates how data can be used to improve customer retention decisions in the streaming services industry through a data-driven approach. Rather than relying on assumptions or intuition to determine which users are likely to cancel their subscriptions, the objective of this analysis is to predict which customers are at risk of churn and to identify the key behavioral and subscription-related factors that influence that decision.

For example, managers of streaming platforms may assume that users with infrequent usage are more likely to cancel their subscriptions. While assumptions like this are reasonable, they are often not supported by a structured analytical framework that quantifies the impact of each factor. This project addresses that gap by applying data science techniques to model customer churn using observable variables such as content consumption, watch time, inactivity patterns, subscription pricing, tenure, and account usage characteristics.

Instead of relying on broad assumptions such as "users who watch less content are more likely to churn," the model produces a clear classification of churn risk based on interpretable decision rules. A decision tree approach is used to identify how combinations of user behaviors and subscription characteristics interact to influence the likelihood that a customer will remain subscribed or cancel their service. This makes the results transparent, intuitive, and directly applicable in a business setting.

The analysis is based on a dataset of 1,000 streaming service users, where each observation represents an individual subscriber. The dataset includes variables related to user engagement (such as watch hours and content views), inactivity (days since last login), subscription details (monthly fee and plan type), and customer tenure. Prior to modeling, the data was prepared by handling categorical variables, removing non-informative identifiers, and partitioning the dataset into training, validation, and test sets. A decision tree classification model was then developed and further refined through hyperparameter tuning to improve predictive performance. The model was evaluated using confusion matrices and standard classification metrics, including accuracy, precision, recall, and F1-score.

This project provides predictive insight and practical business value. The model enables streaming platforms to identify users at higher risk of churn and to prioritize retention strategies more effectively. Additionally, the analysis highlights the critical role of user engagement in driving retention, demonstrating that customers who actively consume content are significantly less likely to cancel their subscriptions. Although this study focuses on streaming services, the same decision tree-based framework can be applied to other subscription-based businesses where customer retention is a key objective.

BUSINESS CONTEXT & PROBLEM DEFINITION

In many services, customer retention decisions are often influenced by intuition, prior experience, or general assumptions about user behavior. While such an approach may be manageable for smaller platforms, it becomes increasingly inefficient as companies grow their subscriber base and operate in highly competitive markets. Customer churn is frequently addressed reactively, after users have already canceled their subscriptions, rather than proactively identifying at-risk customers in advance.

This challenge is particularly relevant in subscription-based digital services such as streaming platforms, where users can easily switch between providers with minimal friction. Companies often assume that customers are more likely to cancel due to factors such as low engagement, high subscription costs, or infrequent platform usage. While these assumptions are reasonable, they should be supported by a structured analytical framework that quantifies the impact of each factor or explains how multiple factors interact to influence churn behavior. As a result, retention strategies may be applied broadly and inefficiently, targeting large groups of users instead of focusing on those at the highest risk of cancellation.

Management requires an objective and data-driven method to assess churn risk before customers decide to leave, in order to allocate retention resources more effectively. Without such an approach, companies risk investing time and incentives in users who would remain subscribed anyway, while failing to intervene early with those most likely to churn.

The central question guiding this analysis is therefore: Can user behavior and subscription data be used to reliably predict customer churn, and which factors most strongly influence a user's decision to remain subscribed or cancel? By addressing this question, this document aims to provide a practical decision-support framework that enables streaming platforms to proactively manage retention risk and improve customer lifetime value.

DATA DESCRIPTION & PREPARATION

The analysis is based on a customer-level dataset of 1,000 streaming service users, where each record represents an individual subscriber. The dataset reflects user behavior within a digital streaming platform, capturing differences in engagement patterns, subscription characteristics, and account usage. The dataset includes the following categories of information:

A) Target Variable

- **churn**: Binary outcome variable indicating subscription status
  1 = Churned (customer canceled subscription)
  0= Stayed (customer remained subscribed)

B) User Engagement Variables

These variables capture how actively users interact with the platform and consume content:

- **watch_hours_per_week**: Average number of hours the user watches content per week
- **content_views_30d**: Number of titles viewed in the past 30 days

C) Inactivity and Usage Behavior

These variables reflect how recent users interact with the platform:

- **days_since_last_login**: Number of days since the user last accessed the platform

D) Subscription and Pricing Variables

These variables describe the user's subscription plan and cost structure:

- **monthly_fee**: Monthly subscription fee paid by the user
- **plan_type**: Subscription tier (Basic, Standard, Premium)

E) Customer Tenure and Account Characteristics

These variables describe user history and account structure:

- **tenure_months**: Length of time the user has been subscribed to the platform
- **num_profiles**: Number of profiles associated with the account

The dataset was reviewed to confirm that all variables were in appropriate formats. The categorical variable was encoded using one-hot encoding to allow its use in the decision tree model. The dataset was partitioned into training (50%), validation (30%), and test (20%) sets using stratified sampling to preserve the distribution of the target variable.

Header of the dataset:

customer_id	monthly_fee	watch_hours_per_week	days_since_last_login	tenure_months	num_profiles
STR0001	14.9	10.2	1	20	2
STR0002	21.83	9	3	6	2
STR0003	15.51	16.9	0	26	2
STR0004	14.06	5.9	11	36	3
STR0005	10.61	7.7	6	17	1

plan_type	content_views_30d	churn
Standard	14	0
Premium	14	0
Standard	25	0
Standard	10	1
Basic	12	0

METHODOLOGY

As discussed earlier, the objective of this analysis is to predict whether a streaming service user is likely to cancel their subscription. The approach estimates the likelihood of churn for each user based on observable engagement, usage behavior, subscription characteristics, and account-related factors. These likelihood estimates allow streaming platforms to proactively identify at-risk users and design targeted retention strategies before customers cancel their subscriptions.

1) Information used to predict customer churn

The analysis relies exclusively on user-level information that is routinely available to streaming platforms and directly relevant to churn behavior. These variables capture engagement intensity, inactivity patterns, pricing sensitivity, and customer loyalty.

- Watch hours per week,
- Number of content views in the last 30 days,
- Days since last login,
- Monthly subscription fee,
- Subscription plan type (Basic, Standard, Premium),
- Length of time the user has been subscribed (tenure),
- Number of profiles associated with the account.

Together, these variables reflect the key drivers of churn in streaming services: engagement level, perceived value, inactivity, and customer loyalty.

2) Preparing the data for fair and reliable analysis

Before building predictive models, several preparation steps were performed to ensure accuracy, consistency, and interpretability:

- The **customer_id** variable was removed, as it represents a unique identifier and does not contribute to prediction.
- Categorical attributes (such as subscription plan type) were converted into indicator variables using one-hot encoding.
- Numeric variables were retained in their original scale to preserve business interpretability.
- The dataset was divided into **training, validation, and test sets** using stratified sampling to maintain the same churn distribution across all subsets.
- Feature alignment was applied after encoding to ensure consistency across all datasets.

These steps ensure that the model captures meaningful behavioral patterns rather than artifacts of data formatting.

3) Predictive modeling approach: Decision Tree model

A decision tree classification model was selected as the analytical approach. This method reflects how business decisions are often made in practice, through a sequence of logical rules based on user behavior and subscription characteristics.

The model segments users into progressively smaller groups based on factors such as content consumption, pricing, tenure, and inactivity. Each final group is associated with a probability of churn, allowing the business to understand both which users are at risk and the specific drivers behind that risk.

To improve model performance and generalization, hyperparameter tuning was applied to control tree depth and prevent overfitting while maintaining interpretability.

4) Model evaluation

Model performance was evaluated across training, validation, and test datasets to ensure reliability and generalizability. The evaluation metrics include:

- Accuracy: Measures overall correctness of predictions
- Precision: Indicates how often users predicted to churn actually cancel
- Recall: Measures the model's ability to correctly identify churned users
- F1-score: Balances precision and recall

Confusion matrices were also analyzed to better understand prediction errors, particularly missed churn cases. This multi-metric evaluation ensures that the model supports effective retention strategies while balancing cost and accuracy.

5) Variable importance and model refinement

To ensure actionable insights, the analysis examined which variables most strongly influence churn predictions. After applying hyperparameter tuning, variables related to content consumption, subscription pricing, and customer tenure emerged as the most influential drivers.

6) Rules and decision logic

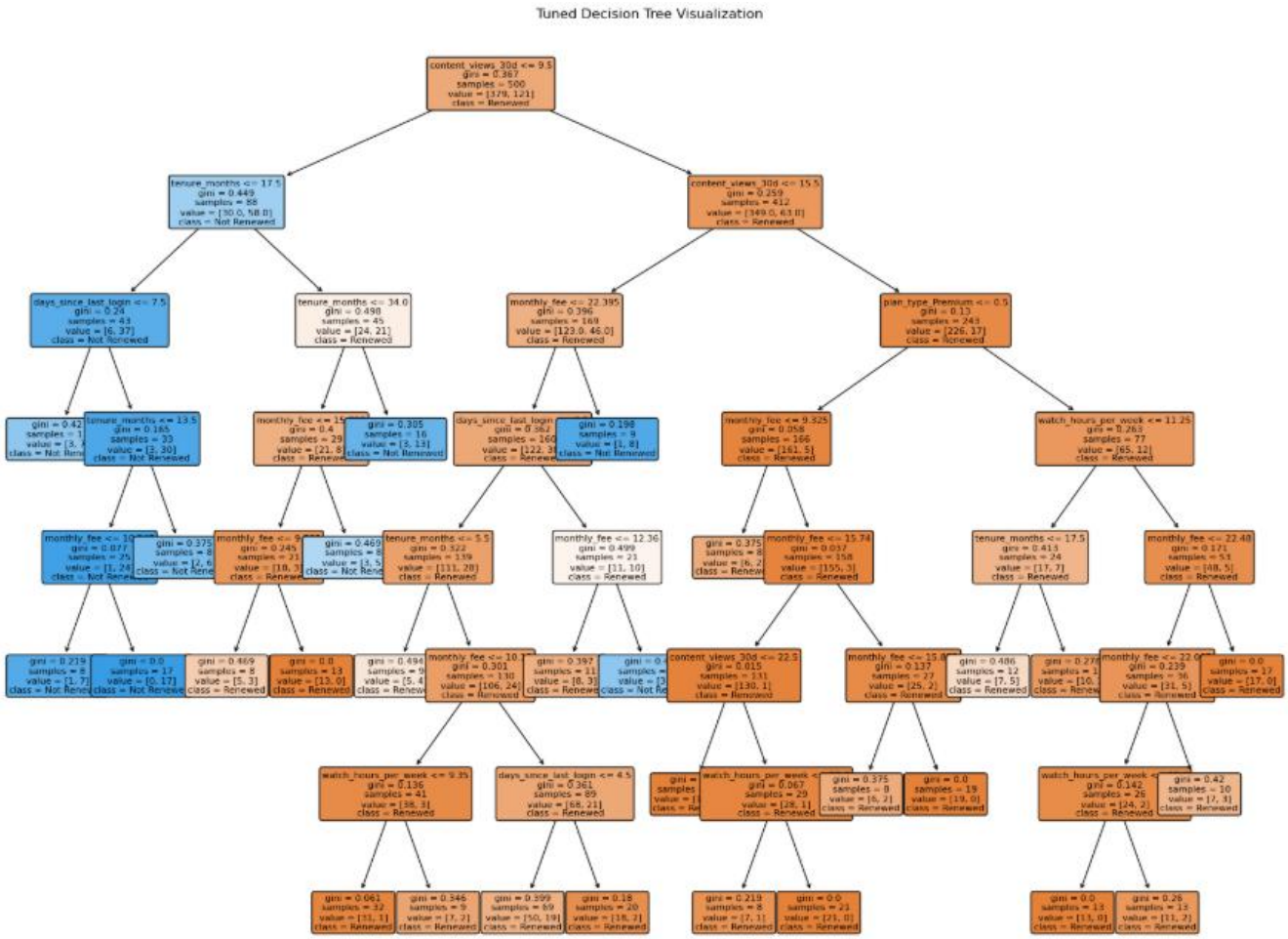
A key advantage of the decision tree approach is its ability to express predictions as clear and interpretable decision rules. After tuning the model to control complexity, the final tree produces a manageable set of rules describing how combinations of user behaviors influence churn risk.

Each rule represents a distinct user segment defined by factors such as engagement level, inactivity, pricing, and tenure. For each segment, the model estimates the probability of churn, allowing the business to understand both who is at risk and why.

These rule-based insights support transparent decision-making and can be directly translated into operational actions, such as targeted retention campaigns, personalized recommendations, or pricing adjustments. By presenting predictions in an interpretable format, the model bridges the gap between data science and practical business application.

ANALYSIS & MODEL RESULTS

The decision tree provides a clear and intuitive representation of how the model predicts customer churn. It operates as a structured decision process: starting with the full set of users and progressively splitting them into smaller groups based on the variables that best distinguish between customers who remain subscribed and those who churn. Each split is based on a simple and interpretable condition, such as the level of content consumption, subscription price, or user inactivity. By following these splits, the model ultimately forms distinct customer segments, each associated with a different probability of churn.



A key observation from the decision tree is that user engagement plays a dominant role in predicting churn behavior. The first and most important split is based on the number of content views in the last 30 days, indicating that recent content consumption is the strongest factor separating retained users from those at risk. Users with low content consumption are significantly more likely to churn, while highly engaged users tend to remain subscribed.

Feature importance analysis further reinforces these findings by quantifying the contribution of each variable to the model's predictions. The results show that:

- **content_views_30d** is the most influential variable, accounting for over half of the model's predictive power
- **monthly_fee** and **tenure_months** are also significant contributors
- **days_since_last_login** and **watch_hours_per_week** play a secondary role
- Variables such as **num_profiles** and some **plan indicators** have minimal or no impact

These results indicate that **deep engagement with content is the primary driver of customer retention**, while pricing and customer tenure provide additional explanatory power. In contrast, variables with low importance contribute little to prediction and can be deprioritized in business decision-making.

```
Feature Importances for Tuned Decision Tree:
content_views_30d        0.523601
monthly_fee              0.205052
tenure_months            0.175207
days_since_last_login    0.050544
watch_hours_per_week     0.026403
plan_type_Premium        0.019193
num_profiles             0.000000
plan_type_Standard       0.000000
```

Interpretation of Decision Rules

The decision rules generated by the model represent the most practical and actionable output. Each rule defines a specific customer segment based on a combination of conditions, such as engagement level, subscription cost, tenure, and activity patterns. At the end of each rule, the model provides probabilities indicating how likely users in that segment are to remain subscribed or churn.

For example, one rule identifies users with low content consumption and shorter tenure as having a significantly higher probability of churn, even if they have logged in recently. This suggests that simply accessing the platform is not sufficient, users must actively consume content to perceive value.

Conversely, another rule highlights that highly engaged users who watch a large number of titles and spend more time on the platform are very unlikely to churn, even when paying higher subscription fees. This indicates that perceived value from content consumption outweighs price sensitivity for engaged users.

The rules are the most practical output of the model. Each rule describes a specific customer profile (a segment) created by a combination of conditions, such as pricing level change, complaint history,

payment patterns, and tenure. At the end of each rule, the model provides probabilities showing how often customers in that segment historically renewed versus not renewed. This allows the business to interpret the model not as a black box, but as a set of understandable customer segments with measurable risk levels that can be targeted with retention actions.

Example:

Rule: content_views_30d > 9.50 AND plan_type_Premium > 0.50 AND watch_hours_per_week > 11.25 AND monthly_fee > 22.09 -> Probabilities: Renewed: 0.7000, Not Renewed: 0.3000

This rule describes customers who are highly engaged with the platform, as indicated by a large number of content views and high weekly watch time, and who are subscribed to a Premium plan with a relatively high monthly fee. Despite paying more, these users actively consume content and derive value from the service. As a result, they are more likely to remain subscribed, with a 70% probability of renewal. However, the presence of a 30% churn probability suggests that even among highly engaged Premium users, price sensitivity or alternative options may still influence some customers to cancel.

The tuned decision tree produced a large number of individual decision rules (approximately 28) because it separates customers into many smaller segments, each with distinct renewal patterns and risk profiles. This level of detail allows the model to capture subtle differences in customer behavior. Below are a few representative examples of the decision rules identified by the model.

Logical rules and probabilities for each leaf of the tuned decision tree:

Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login <= 7.50 -> Probabilities: Renewed: 0.3000, Not Renewed: 0.7000

Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login > 7.50 AND tenure_months <= 13.50 AND monthly_fee <= 10.25 -> Probabilities: Renewed: 0.1250, Not Renewed: 0.8750

Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login > 7.50 AND tenure_months <= 13.50 AND monthly_fee > 10.25 -> Probabilities: Renewed: 0.0000, Not Renewed: 1.0000

Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login > 7.50 AND tenure_months > 13.50 -> Probabilities: Renewed: 0.2500, Not Renewed: 0.7500

Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months <= 34.00 AND monthly_fee <= 15.72 AND monthly_fee <= 9.78 -> Probabilities: Renewed: 0.6250, Not Renewed: 0.3750

Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months <= 34.00 AND monthly_fee <= 15.72 AND monthly_fee > 9.78 -> Probabilities: Renewed: 1.0000, Not Renewed: 0.0000

Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months <= 34.00 AND monthly_fee > 15.72 -> Probabilities: Renewed: 0.3750, Not Renewed: 0.6250

Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months > 34.00 -> Probabilities: Renewed: 0.1875, Not Renewed: 0.8125

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months <= 5.50 -> Probabilities: Renewed: 0.5556, Not Renewed: 0.4444

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND monthly_fee <= 10.18 AND watch_hours_per_week <= 9.35 -> Probabilities: Renewed: 0.9688, Not Renewed: 0.0312

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND monthly_fee <= 10.18 AND watch_hours_per_week > 9.35 -> Probabilities: Renewed: 0.7778, Not Renewed: 0.2222

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND monthly_fee > 10.18 AND days_since_last_login <= 4.50 -> Probabilities: Renewed: 0.7246, Not Renewed: 0.2754

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND monthly_fee > 10.18 AND days_since_last_login > 4.50 -> Probabilities: Renewed: 0.9000, Not Renewed: 0.1000

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login > 8.50 AND monthly_fee <= 12.36 -> Probabilities: Renewed: 0.7273, Not Renewed: 0.2727

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login > 8.50 AND monthly_fee > 12.36 -> Probabilities: Renewed: 0.3000, Not Renewed: 0.7000

Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee > 22.40 -> Probabilities: Renewed: 0.1111, Not Renewed: 0.8889

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium <= 0.50 AND monthly_fee <= 9.32 -> Probabilities: Renewed: 0.7500, Not Renewed: 0.2500

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium <= 0.50 AND monthly_fee > 9.32 AND monthly_fee <= 15.74 AND content_views_30d <= 22.50 -> Probabilities: Renewed: 1.0000, Not Renewed: 0.0000

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium <= 0.50 AND monthly_fee > 9.32 AND monthly_fee <= 15.74 AND content_views_30d > 22.50 AND watch_hours_per_week <= 13.60 -> Probabilities: Renewed: 0.8750, Not Renewed: 0.1250

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium <= 0.50 AND monthly_fee > 9.32 AND monthly_fee <= 15.74 AND content_views_30d > 22.50 AND watch_hours_per_week > 13.60 -> Probabilities: Renewed: 1.0000, Not Renewed: 0.0000

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium <= 0.50 AND monthly_fee > 9.32 AND monthly_fee > 15.74 AND monthly_fee <= 15.88 -> Probabilities: Renewed: 0.7500, Not Renewed: 0.2500

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium <= 0.50 AND monthly_fee > 9.32 AND monthly_fee > 15.74 AND monthly_fee > 15.88 -> Probabilities: Renewed: 1.0000, Not Renewed: 0.0000

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium > 0.50 AND watch_hours_per_week <= 11.25 AND tenure_months <= 17.50 -> Probabilities: Renewed: 0.5833, Not Renewed: 0.4167

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium > 0.50 AND watch_hours_per_week <= 11.25 AND tenure_months > 17.50 -> Probabilities: Renewed: 0.8333, Not Renewed: 0.1667

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium > 0.50 AND watch_hours_per_week > 11.25 AND monthly_fee <= 22.48 AND monthly_fee <= 22.09 AND watch_hours_per_week <= 12.60 -> Probabilities: Renewed: 1.0000, Not Renewed: 0.0000

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium > 0.50 AND watch_hours_per_week > 11.25 AND monthly_fee <= 22.48 AND monthly_fee <= 22.09 AND watch_hours_per_week > 12.60 -> Probabilities: Renewed: 0.8462, Not Renewed: 0.1538

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium > 0.50 AND watch_hours_per_week > 11.25 AND monthly_fee <= 22.48 AND monthly_fee > 22.09 -> Probabilities: Renewed: 0.7000, Not Renewed: 0.3000

Rule: content_views_30d > 9.50 AND content_views_30d > 15.50 AND plan_type_Premium > 0.50 AND watch_hours_per_week > 11.25 AND monthly_fee > 22.48 -> Probabilities: Renewed: 1.0000, Not Renewed: 0.0000

The results indicate that the model performs well and is appropriate for practical business use. It demonstrates a strong ability to learn patterns from historical user behavior and applies that knowledge consistently when evaluating new customers. Across unseen validation and test data,

the model correctly classifies approximately 79–84% of users, reflecting a reliable and realistic level of accuracy for predicting customer churn in a subscription-based setting.

When the model identifies a user as being at risk of churn, it is correct a majority of the time, indicating reasonable precision. At the same time, the model achieves a balanced improvement in recall, successfully identifying around half of the users who ultimately cancel their subscriptions. This represents a meaningful enhancement compared to a more conservative model, as it allows the business to detect a larger share of at-risk customers.

Importantly, the model maintains a practical balance between precision and recall. While it may generate some false positives, this trade-off is often acceptable in customer retention contexts where early intervention can prevent churn. By capturing a broader set of potentially at-risk users without overwhelming the system with excessive false alarms, the model enables more effective and targeted retention strategies.

The model provides a dependable and actionable foundation for supporting retention decisions, prioritizing outreach efforts, and improving customer management strategies, while remaining interpretable and aligned with real-world business needs.

```
Best hyperparameters: {'max_depth': 7, 'min_samples_leaf': 8, 'min_samples_split': 19}

Confusion Matrix - Tuned Training Set:
[[363  16]
 [ 51  70]]

Confusion Matrix - Tuned Validation Set:
[[209  18]
 [ 30  43]]

Confusion Matrix - Tuned Test Set:
[[134  18]
 [ 24  24]]

Tuned Training Set Metrics:
  Accuracy: 0.8660
  Precision: 0.8140
  Recall: 0.5785
  F1-Score: 0.6763

Tuned Validation Set Metrics:
  Accuracy: 0.8400
  Precision: 0.7049
  Recall: 0.5890
  F1-Score: 0.6418

Tuned Test Set Metrics:
  Accuracy: 0.7900
  Precision: 0.5714
  Recall: 0.5000
  F1-Score: 0.5333
```

BUSINESS IMPLICATIONS & RECOMMENDATIONS

This document demonstrates how a streaming platform can transition from reactive, intuition-based retention efforts to a proactive, data-driven approach to managing customer churn. Rather than treating all users the same, the decision tree model evaluates each subscriber's behavior and subscription characteristics to estimate the likelihood of cancellation.

The results show that user engagement, particularly recent content consumption, is the most important driver of retention, followed by subscription pricing and customer tenure. Users who consume little content, exhibit lower engagement, or face higher subscription costs relative to their usage are significantly more likely to churn. In contrast, highly engaged users who frequently watch content and maintain consistent platform usage tend to remain subscribed, even at higher price levels. These insights allow the business to focus retention strategies on the factors that matter most, rather than relying on broad or untargeted interventions.

From an operational perspective, the model supports different retention strategies depending on business priorities. A targeted (conservative) strategy would focus on users identified as high risk, allowing the platform to concentrate on retention resources, such as personalized recommendations, promotional offers, or re-engagement campaigns, on a smaller group where intervention is most needed. This approach helps control costs while protecting the most vulnerable segment of the user base.

A more proactive strategy would involve reaching out to both high-risk and moderate-risk users. While this may include some users who would not have churned, it increases the likelihood of preventing avoidable cancellations. In subscription-based businesses, the cost of targeted interventions, such as notifications, email campaigns, or content recommendations, is often relatively low compared to the long-term value of retaining a customer, making this approach particularly effective.

The model produces clear, rule-based explanations based on factors. These rules are easy for business teams to understand and act upon, making the model suitable for use by marketing, product, and customer experience teams without requiring advanced technical expertise. The model balances predictive performance with interpretability, ensuring it can be both effective and practical in real-world applications.

The recommended approach is to use the tuned decision tree model as a decision-support tool for churn management and retention planning. By prioritizing users based on predicted churn risk and focusing on the key drivers identified by the model, streaming platforms can improve customer retention, allocate resources more efficiently, and design more personalized user experiences. Over time, the model can be updated with new data to reflect evolving user behavior, content trends, and pricing strategies, ensuring continued relevance and effectiveness.

Appendix / Python codes used for this analysis.

```
# read data
df = pd.read_csv('streamingv2.csv')
```

```
# how many rows and columns
df.shape
```

```
(1000, 9)
```

```
# Preview
print(df.head())
```

```
  customer_id  monthly_fee  watch_hours_per_week  days_since_last_login  \
0     STR0001        14.90                  10.2                      1
1     STR0002        21.83                   9.0                      3
2     STR0003        15.51                  16.9                      0
3     STR0004        14.06                   5.9                     11
4     STR0005        10.61                   7.7                      6

   tenure_months  num_profiles plan_type  content_views_30d  churn
0             20             2  Standard                 14      0
1              6             2   Premium                 14      0
2             26             2  Standard                 25      0
3             36             3  Standard                 10      1
4             17             1     Basic                 12      0
```

```
# list the columns with the data types
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   customer_id            1000 non-null   object
 1   monthly_fee            1000 non-null   float64
 2   watch_hours_per_week   1000 non-null   float64
 3   days_since_last_login  1000 non-null   int64
 4   tenure_months          1000 non-null   int64
 5   num_profiles           1000 non-null   int64
 6   plan_type              1000 non-null   object
 7   content_views_30d      1000 non-null   int64
 8   churn                  1000 non-null   int64
dtypes: float64(2), int64(5), object(2)
memory usage: 70.4+ KB
None
```

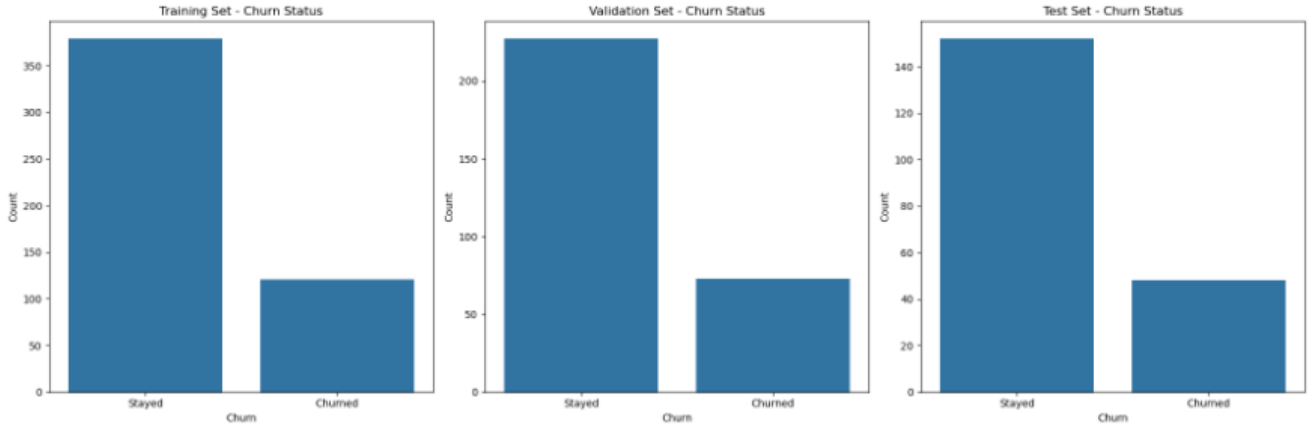
```
# Look at counts of 0 and 1 values for policy renewal status

df = df.drop('customer_id', axis=1)

churn_counts = df['churn'].value_counts()

print("churn counts (0s and 1s):")
print(churn_counts)
```

```
churn counts (0s and 1s):
churn
0    758
1    242
Name: count, dtype: int64
```

```python
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns

# Define the target variable
target = 'churn'

# Define features and target
X = df.drop(target, axis=1)
y = df[target]

# First split: 80% (train+val) and 20% test
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Second split: from the 80%, create 50% train and 30% validation
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.375, random_state=42, stratify=y_temp
)

# Print shapes
print("Training set shape:", X_train.shape, y_train.shape)
print("Validation set shape:", X_val.shape, y_val.shape)
print("Test set shape:", X_test.shape, y_test.shape)
```

```
Training set shape: (500, 7) (500,)
Validation set shape: (300, 7) (300,)
Test set shape: (200, 7) (200,)
```

```python
# Combine features and target for plotting
train_data = X_train.copy()
train_data['churn'] = y_train

val_data = X_val.copy()
val_data['churn'] = y_val

test_data = X_test.copy()
test_data['churn'] = y_test

# Plot the distribution of churn
fig, axs = plt.subplots(1, 3, figsize=(18, 6))

sns.countplot(data=train_data, x='churn', ax=axs[0])
axs[0].set_title('Training Set - Churn Status')
axs[0].set_xlabel('Churn')
axs[0].set_ylabel('Count')
axs[0].set_xticks([0, 1])
axs[0].set_xticklabels(['Stayed', 'Churned'])

sns.countplot(data=val_data, x='churn', ax=axs[1])
axs[1].set_title('Validation Set - Churn Status')
axs[1].set_xlabel('Churn')
axs[1].set_ylabel('Count')
axs[1].set_xticks([0, 1])
axs[1].set_xticklabels(['Stayed', 'Churned'])

sns.countplot(data=test_data, x='churn', ax=axs[2])
axs[2].set_title('Test Set - Churn Status')
axs[2].set_xlabel('Churn')
axs[2].set_ylabel('Count')
axs[2].set_xticks([0, 1])
axs[2].set_xticklabels(['Stayed', 'Churned'])

plt.tight_layout()
plt.show()
```



Hyperparameter Tuning

```
# Define the parameter grid
param_grid = {
    'max_depth': range(2, 30),
    'min_samples_split': range(2, 30),
    'min_samples_leaf': range(1, 20)
}

print("Parameter grid defined:")
print(param_grid)
```

```
Parameter grid defined:
{'max_depth': range(2, 30), 'min_samples_split': range(2, 30), 'min_samples_leaf': range(1, 20)}
```

```
from sklearn.model_selection import GridSearchCV

# Instantiate GridSearchCV
grid_search = GridSearchCV(
    dt_model,
    param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)

# Fit GridSearchCV to the encoded training data
grid_search.fit(X_train_enc, y_train)

# Print the best hyperparameters and best cross-validation score
print("Best hyperparameters found:", grid_search.best_params_)
print("Best cross-validation F1-score:", grid_search.best_score_)
```

```
Best hyperparameters found: {'max_depth': 7, 'min_samples_leaf': 8, 'min_samples_split': 19}
Best cross-validation F1-score: 0.5484199471174043
```

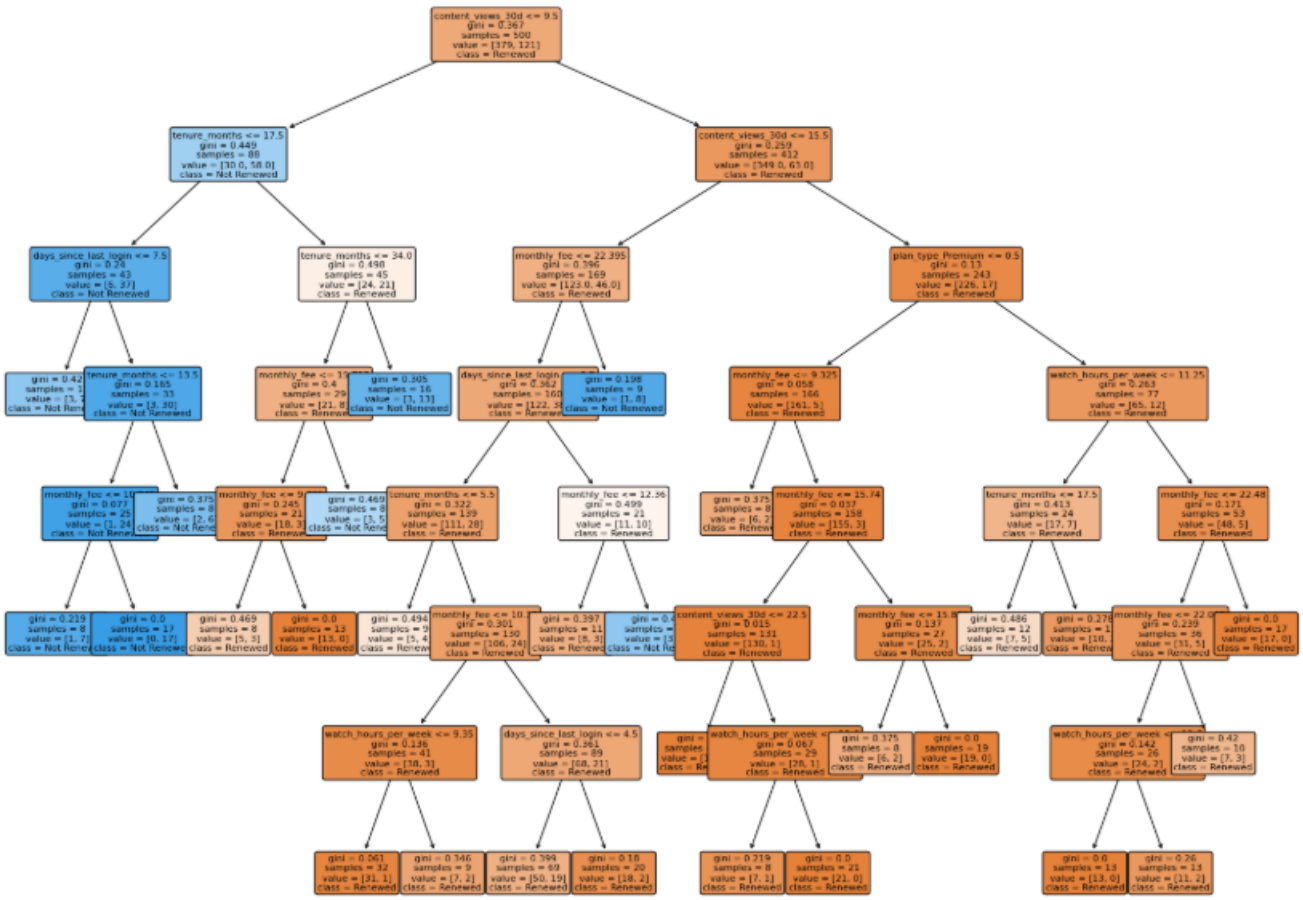
```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

# Retrieve the best hyperparameters
best_params = grid_search.best_params_
print("Best hyperparameters:", best_params)

# Instantiate a new Decision Tree Classifier with the best hyperparameters
tuned_dt_model = DecisionTreeClassifier(**best_params, random_state=42)

# Train the tuned model on the encoded training dataset
tuned_dt_model.fit(X_train_enc, y_train)

# Make predictions on training set
y_train_pred_tuned = tuned_dt_model.predict(X_train_enc)
cm_train_tuned = confusion_matrix(y_train, y_train_pred_tuned)

# Make predictions on validation set
y_val_pred_tuned = tuned_dt_model.predict(X_val_enc)
cm_val_tuned = confusion_matrix(y_val, y_val_pred_tuned)

# Make predictions on test set
y_test_pred_tuned = tuned_dt_model.predict(X_test_enc)
cm_test_tuned = confusion_matrix(y_test, y_test_pred_tuned)

print("\nConfusion Matrix - Tuned Training Set:")
print(cm_train_tuned)

print("\nConfusion Matrix - Tuned Validation Set:")
print(cm_val_tuned)

print("\nConfusion Matrix - Tuned Test Set:")
print(cm_test_tuned)

# Calculate metrics for Training Set
accuracy_train_tuned = accuracy_score(y_train, y_train_pred_tuned)
precision_train_tuned = precision_score(y_train, y_train_pred_tuned)
recall_train_tuned = recall_score(y_train, y_train_pred_tuned)
f1_train_tuned = f1_score(y_train, y_train_pred_tuned)

print("\nTuned Training Set Metrics:")
print(f"  Accuracy: {accuracy_train_tuned:.4f}")
print(f"  Precision: {precision_train_tuned:.4f}")
print(f"  Recall: {recall_train_tuned:.4f}")
print(f"  F1-Score: {f1_train_tuned:.4f}")

# Calculate metrics for Validation Set
accuracy_val_tuned = accuracy_score(y_val, y_val_pred_tuned)
precision_val_tuned = precision_score(y_val, y_val_pred_tuned)
recall_val_tuned = recall_score(y_val, y_val_pred_tuned)
f1_val_tuned = f1_score(y_val, y_val_pred_tuned)
```

```python
# Calculate metrics for Validation Set
accuracy_val_tuned = accuracy_score(y_val, y_val_pred_tuned)
precision_val_tuned = precision_score(y_val, y_val_pred_tuned)
recall_val_tuned = recall_score(y_val, y_val_pred_tuned)
f1_val_tuned = f1_score(y_val, y_val_pred_tuned)

print("\nTuned Validation Set Metrics:")
print(f"  Accuracy: {accuracy_val_tuned:.4f}")
print(f"  Precision: {precision_val_tuned:.4f}")
print(f"  Recall: {recall_val_tuned:.4f}")
print(f"  F1-Score: {f1_val_tuned:.4f}")

# Calculate metrics for Test Set
accuracy_test_tuned = accuracy_score(y_test, y_test_pred_tuned)
precision_test_tuned = precision_score(y_test, y_test_pred_tuned)
recall_test_tuned = recall_score(y_test, y_test_pred_tuned)
f1_test_tuned = f1_score(y_test, y_test_pred_tuned)

print("\nTuned Test Set Metrics:")
print(f"  Accuracy: {accuracy_test_tuned:.4f}")
print(f"  Precision: {precision_test_tuned:.4f}")
print(f"  Recall: {recall_test_tuned:.4f}")
print(f"  F1-Score: {f1_test_tuned:.4f}")
```

```
Best hyperparameters: {'max_depth': 7, 'min_samples_leaf': 8, 'min_samples_split': 19}

Confusion Matrix - Tuned Training Set:
[[363  16]
 [ 51  70]]

Confusion Matrix - Tuned Validation Set:
[[209  18]
 [ 30  43]]

Confusion Matrix - Tuned Test Set:
[[134  18]
 [ 24  24]]

Tuned Training Set Metrics:
  Accuracy: 0.8660
  Precision: 0.8140
  Recall: 0.5785
  F1-Score: 0.6763

Tuned Validation Set Metrics:
  Accuracy: 0.8400
  Precision: 0.7049
  Recall: 0.5890
  F1-Score: 0.6418

Tuned Test Set Metrics:
  Accuracy: 0.7900
  Precision: 0.5714
  Recall: 0.5000
  F1-Score: 0.5333
```

```python
# Plot the tuned decision tree
plot_tree(
    tuned_dt_model,
    feature_names=X_train_enc.columns,
    class_names=['Renewed', 'Not Renewed'],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title('Tuned Decision Tree Visualization')
plt.show()
```

Tuned Decision Tree Visualization



```python
# The number of splits in a decision tree is the number of internal nodes.
# This is equal to the total number of nodes minus the number of leaf nodes.
# We can access the tree structure from the tuned_dt_model.tree_ attribute.

tree_ = tuned_dt_model.tree_
total_nodes = tree_.node_count
leaf_nodes = tree_.n_leaves

number_of_splits = total_nodes - leaf_nodes

print(f"The tuned decision tree has {number_of_splits} splits (internal nodes).")
print(f"The tuned decision tree has {leaf_nodes} leaves.")
```

```
The tuned decision tree has 27 splits (internal nodes).
The tuned decision tree has 28 leaves.
```

```python
# Function to extract decision rules and class probabilities
def get_decision_path_rules(tree, feature_names, node_id, current_rule, class_names):
    """Recursively gets the decision path rules and probabilities for each leaf node."""
    left_child = tree.children_left[node_id]
    right_child = tree.children_right[node_id]

    # If it's a leaf node
    if left_child == -1 and right_child == -1:
        # Get class counts at the leaf
        value = tree.value[node_id][0]
        total_samples_at_leaf = np.sum(value)

        # Calculate probabilities
        probabilities = value / total_samples_at_leaf

        # Format output
        prob_strings = [
            f"{class_names[i]}: {probabilities[i]:.4f}"
            for i in range(len(class_names))
        ]
        prob_output = ", ".join(prob_strings)

        print(f"Rule: {current_rule} -> Probabilities: {prob_output}")
        return

    # Internal node
    feature_index = tree.feature[node_id]
    threshold = tree.threshold[node_id]
    feature_name = feature_names[feature_index]

    # Left child rule
    left_rule = (
        f"{current_rule} AND {feature_name} <= {threshold:.2f}"
        if current_rule else f"{feature_name} <= {threshold:.2f}"
    )
    get_decision_path_rules(tree, feature_names, left_child, left_rule, class_names)

    # Right child rule
    right_rule = (
        f"{current_rule} AND {feature_name} > {threshold:.2f}"
        if current_rule else f"{feature_name} > {threshold:.2f}"
    )
    get_decision_path_rules(tree, feature_names, right_child, right_rule, class_names)
```

```python
# Get tree structure and correct feature names (encoded)
tree_structure = tuned_dt_model.tree_
feature_names = list(X_train_enc.columns)

# Correct class names for your target
class_names = ['Renewed', 'Not Renewed']

print("Logical rules and probabilities for each leaf of the tuned decision tree:")

# Start recursion from root node
get_decision_path_rules(tree_structure, feature_names, 0, "", class_names)
```

```
Logical rules and probabilities for each leaf of the tuned decision tree:
Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login <= 7.50 -> Probabilities: Renewed: 0.3000, Not Renewed: 0.7000
Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login > 7.50 AND tenure_months <= 13.50 AND monthly_fee <= 10.25 -> Prob
abilities: Renewed: 0.1250, Not Renewed: 0.8750
Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login > 7.50 AND tenure_months <= 13.50 AND monthly_fee > 10.25 -> Proba
bilities: Renewed: 0.0000, Not Renewed: 1.0000
Rule: content_views_30d <= 9.50 AND tenure_months <= 17.50 AND days_since_last_login > 7.50 AND tenure_months > 13.50 -> Probabilities: Renewed: 0.250
0, Not Renewed: 0.7500
Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months <= 34.00 AND monthly_fee <= 15.72 AND monthly_fee <= 9.78 -> Probabilities:
Renewed: 0.6250, Not Renewed: 0.3750
Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months <= 34.00 AND monthly_fee <= 15.72 AND monthly_fee > 9.78 -> Probabilities:
Renewed: 1.0000, Not Renewed: 0.0000
Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months <= 34.00 AND monthly_fee > 15.72 -> Probabilities: Renewed: 0.3750, Not Ren
ewed: 0.6250
Rule: content_views_30d <= 9.50 AND tenure_months > 17.50 AND tenure_months > 34.00 -> Probabilities: Renewed: 0.1875, Not Renewed: 0.8125
Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months <= 5.50 -> P
robabilities: Renewed: 0.5556, Not Renewed: 0.4444
Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND m
onthly_fee <= 10.18 AND watch_hours_per_week <= 9.35 -> Probabilities: Renewed: 0.9688, Not Renewed: 0.0312
Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND m
onthly_fee <= 10.18 AND watch_hours_per_week > 9.35 -> Probabilities: Renewed: 0.7778, Not Renewed: 0.2222
Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND m
onthly_fee > 10.18 AND days_since_last_login <= 4.50 -> Probabilities: Renewed: 0.7246, Not Renewed: 0.2754
Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login <= 8.50 AND tenure_months > 5.50 AND m
onthly_fee > 10.18 AND days_since_last_login > 4.50 -> Probabilities: Renewed: 0.9000, Not Renewed: 0.1000
Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login > 8.50 AND monthly_fee <= 12.36 -> Pro
babilities: Renewed: 0.7273, Not Renewed: 0.2727
Rule: content_views_30d > 9.50 AND content_views_30d <= 15.50 AND monthly_fee <= 22.40 AND days_since_last_login > 8.50 AND monthly_fee > 12.36 -> Prob
abilities: Renewed: 0.3000, Not Renewed: 0.7000
```

```python
import pandas as pd

# Get the feature importances from the tuned decision tree model
feature_importances_tuned_dt = tuned_dt_model.feature_importances_

# Get the feature names from the encoded training data
feature_names_dt = X_train_enc.columns

# Create a pandas Series to view feature importances with names
feature_importance_series_tuned_dt = pd.Series(
    feature_importances_tuned_dt,
    index=feature_names_dt
)

# Sort feature importances in descending order
sorted_feature_importances_tuned_dt = feature_importance_series_tuned_dt.sort_values(ascending=False)

# Print the sorted feature importances
print("Feature Importances for Tuned Decision Tree:")
print(sorted_feature_importances_tuned_dt)
```

```
Feature Importances for Tuned Decision Tree:
content_views_30d        0.523601
monthly_fee              0.205052
tenure_months            0.175207
days_since_last_login    0.050544
watch_hours_per_week     0.026403
plan_type_Premium        0.019193
num_profiles             0.000000
plan_type_Standard       0.000000
dtype: float64
```